LAB_MEETING_PRESENTATION_V4 Korean Speaking Script

LAB_MEETING_PRESENTATION_V4 Korean Speaking Script

Slide 1. A Natural-Language-Driven Quantum Chemistry Calculation Platform Integrating LLM, Model Context Protocol, and PySCF

(구버전 대응: Slide 1)

안녕하세요. 오늘 말씀드릴 내용은 양자화학 계산 자동화 플랫폼입니다. 한 문장으로 먼저 말씀드리면, 이 프로젝트는 실험 화학자가 자연어로 계산을 요청하면, 시스템이 그 요청을 실제 계산 작업으로 바꾸고, 계산 결과를 바로 해석 가능한 시각화 형태로 돌려주는 플랫폼입니다.

이 프로젝트를 단순히 AI 챗봇이라고 부르기는 어렵습니다. 핵심은 말을 그럴듯하게 만드는 것이 아니라, 질문을 실제 계산으로 연결하고, 계산 결과를 다시 사람이 이해할 수 있는 형태로 되돌려주는 전체 연구 워크플로를 자동화했다는 점에 있기 때문입니다.

오늘은 세 가지 질문에 답드리겠습니다. 첫째, 왜 이런 플랫폼이 연구실에서 필요한가. 둘째, 이 시스템은 어떻게 실제 계산까지 연결되는가. 셋째, 이게 앞으로 어떤 연구적 가치와 확장성을 가질 수 있는가. 이 세 가지를 중심으로 말씀드리겠습니다.

Slide 2. Contents

(구버전 대응: 없음 — 새로 추가)

발표 순서를 먼저 말씀드리겠습니다. 크게 네 파트로 구성되어 있습니다. 첫 번째는 Introduction으로, 양자화학 계산이 실험 연구에서 왜 중요한지, 현재 워크플로에 어떤 문제가 있는지, 그리고 왜 지금 이 시도가 가능해졌는지를 다룹니다. 두 번째는 Methods로, 플랫폼의 전체 설계와 각 구성 요소, 그리고 네 단계로 이루어진 실제 구현 파이프라인을 설명드립니다. 세 번째는 Results로, 현재 지원하는 기능과 대표적인 사용 사례, 그리고 실제 워크플로 시연을 보여드립니다. 마지막 네 번째는 Conclusion and Outlook으로, 연구실 차원의 의미, 논문화 방향, 그리고 향후 확장 계획을 정리합니다. 그러면 Introduction부터 시작하겠습니다.

Slide 3. Background: Quantum Chemistry in Experimental Workflows

(구버전 대응: Slide 3 + 4)

먼저 Introduction 섹션입니다. 실험 화학자에게 양자화학 계산이 왜 필요한지부터 짚고 가겠습니다.

결국 계산은 실험에서 바로 보이지 않는 전자 구조 정보를 보여주는 도구입니다. HOMO/LUMO는 반응성이나 전자 이동 방향성을 직관적으로 생각하게 해 주고, geometry optimization은 우리가 보고 있는 구조가 실제로 말이 되는 기준 구조인지 확인하게 해 줍니다. ESP 맵은 분자 표면의 전하 분포를 시각적으로 보여주기 때문에, 어떤 부분이 더 전자 풍부한지, 어떤 부분이 상호작용에 민감한지 빠르게 파악하게 해 줍니다. 중요한 건 이런 계산이 논문 장식용 그림이 아니라는 점입니다. 반응성 해석, 선택성 설명, 기작 가설 정리, 상호작용 예측 같은 연구 질문에 직접 연결됩니다.

그리고 실험자가 계산에서 기대하는 것은 생각보다 거창하지 않을 때가 많습니다. "이 분자의 HOMO를 보여달라", "이 구조를 최적화하면 어떻게 바뀌는지 보고 싶다", "ESP를 보면 어느 쪽이 더 전자 풍부한지 알고 싶다" 같은 질문이 대부분입니다. 출발점은 대형 계산 프로젝트가 아니라, 연구 중간중간 튀어나오는 구체적이고 즉각적인 질문입니다.

Aa

Contents

그런데 현재는 이런 질문조차 실험자가 스스로 계산으로 바로 연결하기 어렵습니다. 그러면 무엇이 이렇게 어렵게 만드는지, 현재 워크플로의 문제를 좀 더 구체적으로 보겠습니다.

Slide 4. Problem Statement: Workflow Fragmentation and Access Barriers

(구버전 대응: Slide 5 + 6)

기존 워크플로는 질문에서 결과까지 너무 많은 단계를 거칩니다. 계산을 부탁할 사람을 찾아야 하고, 구조를 준비해야 하고, 입력 파일을 작성해야 하고, 적절한 계산 조건을 정해야 하고, 계산을 돌린 뒤 다시 결과를 파싱하고 다른 뷰어로 옮겨야 합니다. 그다음에야 비로소 사람이 읽을 수 있는 그림이나 수치가 나옵니다. 이때 문제는 계산 시간만 긴 것이 아닙니다. 구조 준비, 전달, 수정, 재요청, 후처리 같은 주변 단계가 훨씬 더 많은 시간을 잡아먹습니다. 질문이 하나 바뀌면 다시 일부 단계를 반복해야 합니다. 핵심 문제는 계산 엔진이 느리냐 빠르냐보다, 질문과 결과 사이가 너무 많은 수작업과 소프트웨어 전환으로 이어져 있다는 데 있습니다.

이 구조는 연구실 전체 차원에서 병목으로 작동합니다. 계산을 다룰 수 있는 사람이 소수면, 많은 질문이 그 사람에게 몰립니다. 실험자는 기다리게 되고, 계산 전문가는 반복 요청 처리에 시간을 많이 쓰게 됩니다. 장기적으로 더 큰 문제는 계산 리터러시가 넓어지지 않는다는 점입니다. 실험자는 계산을 계속 외주처럼 느끼고, 계산은 연구실 전체의 일상적 도구가 되지 못합니다. 그래서 이 프로젝트는 계산 전문가를 없애려는 것이 아니라, 전문가가 반복 업무보다 더 중요한 검증과 해석에 집중할 수 있게 만드는 구조를 제안합니다. 그러면 이 문제를 왜 지금 풀 수 있게 됐는지, 기술적 배경을 말씀드리겠습니다.

Slide 5. Motivation: Convergence of LLM, MCP, and Open-Source QC Engines

(구버전 대응: Slide 7)

왜 이런 시도가 지금 가능하냐는 질문이 자연스럽게 나옵니다. 세 가지가 동시에 성숙했기 때문입니다. 첫째, PySCF 같은 오픈소스 양자화학 엔진이 실용적 수준으로 충분히 성숙했습니다. 둘째, LLM이 사람이 말한 질문에서 의도를 구조화할 수 있을 정도로 좋아졌습니다. 셋째, 웹 애플리케이션과 도구 오케스트레이션 기술이 발달하면서, 질문 해석과 계산 실행, 결과 시각화를 한 흐름으로 묶는 것이 가능해졌습니다.

과거에도 스크립트 자동화나 GUI 개선은 있었습니다. 하지만 대부분은 어느 한 단계만 조금 더 편하게 만든 수준이었습니다. 지금은 그보다 더 큰 단위, 즉 질문 해석부터 계산 실행, 결과 전달까지 전체 파이프라인을 묶는 것이 현실적인 과제가 됐습니다. 배경, 문제, 기술적 동기를 모두 살펴봤으니, 이제 이 프로젝트가 구체적으로 무엇을 목표로 하는지 정리하겠습니다.

Slide 6. Research Objective and Scope

(구버전 대응: Slide 2 + 8)

이 프로젝트의 목표는 아주 단순하게 정리할 수 있습니다. 양자화학 계산은 중요하지만, 여전히 많은 실험자에게는 접근하기 어려운 도구이고, 저희는 그 접근 경로 자체를 바꾸려는 시도를 하고 있다는 것입니다. 계산을 대신하는 것이 아니라, 계산에 도달하는 문턱을 낮추고, 계산 결과를 더 쉽게 받아볼 수 있게 만드는 것이 본질입니다. "AI가 얼마나 똑똑한가"보다, 실험자가 계산을 더 자주, 더 쉽게, 더 책임 있게 사용할 수 있는 구조를 만들 수 있는가에 초점을 맞추고 있습니다.

좀 더 구체적으로 정의하면, 이것은 "AI가 화학을 대신해 주는 시스템"이 아닙니다. 사용자의 질문을 계산 가능한 작업으로 바꾸고, 실제 계산 엔진을 호출하고, 계산 결과를 다시 사용자 친화적으로 전달하는 연구 워크플로 플랫폼입니다. 앞단에는 자연어 인터페이스, 중간에는 의도 해석과 도구 라우팅, 뒤에는 실제 양자화학 계산 엔진, 마지막에는 시각화와 설명 계층이 있습니다. 이 전체를 하나로 묶어 실험자가 "질문 중심"으로 접근할 수 있게 만든

것이 이 프로젝트의 범위입니다. 그러면 이제 이 플랫폼이 구체적으로 어떻게 설계되고 구현되었는지, Methods 섹션으로 넘어가겠습니다.

Slide 7. Platform Overview and Design Principles

(구버전 대응: Slide 8 — 설계 원칙 부분)

이제 Methods 섹션입니다. 먼저 플랫폼의 전체 설계 원칙부터 말씀드리겠습니다.

이 플랫폼은 몇 가지 핵심 설계 원칙 위에 세워져 있습니다. 첫째, 질문 중심 접근입니다. 사용자가 계산 메뉴를 선택하는 것이 아니라, 자연어 질문으로부터 계산이 시작됩니다. 둘째, 역할 분리입니다. LLM은 의도를 해석하고, MCP 계층은 도구를 연결하며, PySCF가 실제 계산을 수행합니다. 각 역할이 분명히 나뉘어 있어야 결과의 출처와 책임이 명확해집니다. 셋째, 확장 가능한 구조입니다. 이 시스템은 단순한 스크립트가 아니라, 각 부분을 독립적으로 개선하고 교체할 수 있는 플랫폼 구조를 가집니다. 구조 해석을 더 잘하게 만들 수도 있고, 계산 경로를 늘릴 수도 있고, 결과 화면을 더 풍부하게 만들 수도 있습니다.

이런 원칙이 실제 시스템에서 어떤 구조로 구현되는지, 다음 슬라이드에서 전체 아키텍처를 보여드리겠습니다.

Slide 8. System Architecture

(구버전 대응: Slide 14)

전체 구조를 한 번에 보겠습니다. 사용자는 브라우저에서 질문을 합니다. 웹 애플리케이션은 그 질문을 받아 적절한 라우팅 계층으로 보냅니다. 그다음 구조 해석 또는 구조 보강이 필요하면 그 단계를 거치고, 계산 작업이 정해지면 PySCF로 연결됩니다. 계산 결과는 다시 설명과 시각화 계층을 거쳐 브라우저로 돌아옵니다.

사용자 입장에서는 하나의 인터페이스처럼 보이지만, 내부적으로는 역할이 분리되어 있습니다. 이 역할 분리가 중요한 이유는 각 부분을 독립적으로 개선할 수 있기 때문입니다. 이 아키텍처는 크게 세 가지 핵심 구성 요소와 네 단계 구현 파이프라인으로 나뉩니다. 먼저 각 구성 요소가 왜 필요하고 무엇을 하는지 설명드린 뒤, 실제 파이프라인 단계를 순서대로 따라가겠습니다. 첫 번째 구성 요소는 자연어 처리 계층입니다.

Slide 9. Natural Language Processing Layer: Intent Parsing and Task Classification

(구버전 대응: Slide 9)

LLM이 실제로 무엇을 하는지 구체적으로 설명드리겠습니다. 많은 분들이 LLM이라고 하면 그냥 말을 잘하는 모델로 생각하실 수 있습니다. 그런데 이 프로젝트에서 LLM의 역할은 예쁜 문장을 만드는 것이 아닙니다. 핵심 역할은 사용자의 질문을 계산 가능한 형태로 분해하는 것입니다.

예를 들어 "아세톤의 HOMO 보여줘"라는 문장이 들어오면, 시스템은 여기서 최소한 네 가지를 뽑아내야 합니다. 첫째, 대상 분자가 아세톤이라는 것. 둘째, 원하는 작업이 HOMO 계산 또는 오비탈 시각화라는 것. 셋째, 사용자가 원하는 결과 형식이 숫자표가 아니라 시각화라는 것. 넷째, 이전 질문이 있었다면 같은 구조를 이어서 쓰는지 여부입니다.

또 "같은 구조로 ESP도 보고 싶어"라고 하면, 여기서 중요한 건 새 분자를 찾는 게 아니라 이전 문맥을 유지한 채 계산 종류만 바뀌었다는 것을 이해하는 것입니다. 그러니까 LLM은 계산 자체를 하는 게 아니라, 사람 언어를 읽고 분자, 작업, 맥락, 부족한 정보, 기대 결과 형식을 구조화하는 역할을 합니다. 그런데 바로 여기서 중요한 문제가 생깁니다.

Slide 10. Addressing LLM Hallucination Through Structured Tool Invocation

(구버전 대응: Slide 10)

LLM이 질문을 잘 읽는다고 해서, 그 자체로 연구 도구가 되는 것은 아닙니다. 왜냐하면 LLM은 계산을 하지 않아도 마치 계산한 것처럼 그럴듯하게 말할 수 있기 때문입니다. 즉, 값을 할루시네이션으로 대답해 줄 위험이 있습니다.

예를 들어 "이 분자의 HOMO 에너지가 얼마다", "ESP 분포는 이런 식이다"라고 말하는 것은, 실제 계산 없이도 텍스트 모델이 그럴듯하게 흉내 낼 수 있습니다. 하지만 연구에서는 그게 가장 위험합니다. 계산을 안 했는데 계산한 것처럼 보이면 안 되기 때문입니다. 그래서 이 프로젝트에서 중요한 건 LLM이 똑똑하다는 점이 아니라, LLM 단독 답변으로 끝나지 않게 만들었다는 점입니다.

정리하면, 이 시스템은 LLM이 값을 지어내는 시스템이 아니라, LLM이 실제 계산 도구를 호출하도록 설계된 구조를 가진 시스템입니다. 그러면 이 도구 호출 구조를 어떻게 표준화했는지, MCP에 대해 말씀드리겠습니다.

Slide 11. Model Context Protocol: Architecture and Role in Tool Orchestration

(구버전 대응: Slide 11 + 12)

MCP를 정의부터 바로 들어가기보다, 다른 분야 사례부터 말씀드리는 게 이해가 쉽습니다. 요즘은 LLM이 PowerPoint를 조작하거나, Blender에서 모델링 작업을 하거나, 문서를 읽고 정리하거나, 코드를 실행하는 데 연결되는 사례들이 많이 나오고 있습니다. 다시 말해, LLM이 단순히 말만 하는 것이 아니라, 외부 도구를 실제로 다루는 방향으로 발전하고 있습니다. 이런 흐름에서 MCP는 쉽게 말하면, LLM이 외부 도구를 정해진 방식으로 안전하게 호출할 수 있게 해주는 연결 규칙입니다.

그런데 양자화학 쪽은 아직 이 연결이 충분히 정리되어 있지 않습니다. 다른 분야에서는 LLM이 도구를 부리는 구조가 점점 늘어나고 있는데, 양자화학은 여전히 입력 파일 작성, 계산 실행, 결과 파싱, 시각화가 분리된 경우가 많습니다. 그래서 이 프로젝트는 이 흐름을 양자화학으로 가져와, MCP 스타일 도구 연결 구조를 선제적으로 구현해 보는 시도입니다.

이게 바로 할루시네이션 문제와도 연결됩니다. 도구 호출 구조가 분명하면, 어느 순간 구조 조회가 일어났고, 어느 순간 PySCF 계산이 실행됐고, 어느 순간 결과가 패키징됐는지가 분명해집니다. 이건 단순 편의성 문제가 아니라, 연구 도구의 신뢰성 문제입니다. 그러면 이 구조에서 실제 계산을 담당하는 백엔드에 대해 말씀드리겠습니다.

Slide 12. Computational Backend: PySCF Integration and Provenance Tracking

(구버전 대응: Slide 13)

자연어 인터페이스가 아무리 좋아도, 뒤에서 실제 전자구조 계산이 수행되지 않으면 이 시스템은 연구 도구가 아니라 계산을 흉내 내는 인터페이스에 불과합니다.

PySCF는 이 프로젝트에서 실제 계산을 담당하는 과학적 백엔드입니다. Orbital preview, ESP, partial charge analysis, geometry optimization 같은 핵심 계산은 결국 여기서 수행됩니다. 즉, 저희 시스템의 구조는 "AI가 계산해 준다"가 아니라, PySCF 같은 실제 계산 엔진에 더 쉽게, 더 유연하게 접근하게 해 준다는 구조입니다.

역할을 분명히 나눠 보면, LLM은 질문을 읽고 정리합니다. MCP 스타일 연결 계층은 적절한 도구를 부르게 합니다. 하지만 결과를 만들어내는 것은 결국 계산 엔진입니다. 이 분리가 있어야 계산 결과의 출처와 책임이 분명해집니다. 이제 세 가지 구성 요소를 모두 설명드렸으니, 이것들이 실제로 어떤 순서로 작동하는지, 네 단계 구현 파이프라인을 하나씩 따라가겠습니다.

Slide 13. Implementation — Request Understanding and Task Decomposition

(구버전 대응: Slide 15)

구현 파이프라인의 첫 번째 단계는 사용자의 요청을 이해하는 것입니다. 여기서 시스템은 분자 이름이 무엇인지, 원하는 작업이 HOMO인지 ESP인지 optimization인지, 이전 질문과 이어지는지, 아직 정보가 부족한지 등을

판단합니다.

이 단계가 중요한 이유는 자연어 질문이 원래 모호하기 때문입니다. 사람끼리는 자연스럽게 이해되는 문장도, 계산 파이프라인으로 바로 넣으려면 훨씬 더 명확한 구조가 필요합니다. 따라서 이 단계의 핵심은 예쁜 대답을 만드는 것이 아니라, 계산 가능한 구조화된 요청으로 바꾸는 것입니다. 요청이 구조화되면, 그다음은 계산할 분자 구조를 준비하는 단계입니다.

Slide 14. Implementation — Molecular Structure Resolution

(구버전 대응: Slide 16)

두 번째 단계는 구조를 계산 가능한 형태로 준비하는 것입니다. 사실 자동화 시스템에서 가장 어려운 부분 중 하나가 바로 여기입니다. 분자 이름이 들어왔다고 해서 바로 계산 가능한 3D 구조가 손에 쥐어지는 것은 아닙니다. 경우에 따라 외부 구조 소스와 연결해야 하고, 전하 상태나 이온쌍 여부를 따져야 하고, 후보 구조 중 어느 것을 쓸지 판단해야 할 수도 있습니다.

즉, 계산 자동화는 계산 버튼을 누르는 문제만이 아닙니다. 무엇을 계산할 것인지의 전제 조건을 세우는 문제이기도 합니다. 이 구조 준비 단계가 중요한 이유는, 이 단계를 제대로 다루지 않으면 뒤 계산이 아무리 좋아도 결과를 신뢰하기 어렵기 때문입니다. 구조가 준비되면, 그다음은 어떤 계산 경로를 탈지 결정하는 단계입니다.

Slide 15. Implementation — Tool Routing and Calculation Execution

(구버전 대응: Slide 17)

세 번째 단계에서는 어떤 계산 경로를 탈지 결정합니다. 사용자가 오비탈을 원하면 그에 맞는 출력이 필요하고, ESP를 원하면 표면 데이터가 필요하며, optimization이면 구조 갱신 결과가 필요합니다. 같은 분자라도 질문에 따라 필요한 계산 경로와 결과 포맷이 달라집니다.

이 단계가 바로 MCP 스타일 연결 계층과 맞물리는 부분입니다. LLM이 구조화한 요청을 바탕으로, 시스템은 이제 어떤 도구를 실제로 호출할지 결정합니다. 구조 조회 도구를 부를지, PySCF 계산을 돌릴지, 후속 결과 패키징을 할지 정합니다. 이 덕분에 사용자는 내부 계산 메뉴를 다 알지 못해도 됩니다. 질문 중심 인터페이스가 계산 선택 인터페이스를 대신하는 구조가 만들어지는 것입니다. 계산이 실행되면, 마지막으로 결과를 사용자에게 전달하는 단계가 남습니다.

Slide 16. Implementation — Result Packaging and Visualisation

(구버전 대응: Slide 18)

계산이 끝났다고 해서 자동으로 사람이 이해할 수 있는 결과가 되는 것은 아닙니다. 적절한 파일, 표면 데이터, 요약 정보, 설명 문장, 시각화 객체로 다시 묶어야 합니다. 이것이 네 번째 단계입니다.

이 플랫폼은 결과를 파일 시스템 안에 묻어두는 것이 아니라, 브라우저에서 바로 보고 토론할 수 있는 형태로 돌려주는 것을 중요하게 봅니다. 결국 연구자는 숫자 덩어리보다 보이고, 비교되고, 설명 가능한 결과를 원합니다. 바로 이 지점에서 기존 방식과 차이가 커집니다. 기존에는 Gaussian, ORCA, GaussView, Multiwfn처럼 여러 도구와 다양한 파일 포맷을 오가며 학습 곡선을 견뎌야 했다면, 여기서는 결과가 브라우저 안에서 바로 이어집니다. 사용성 측면에서 가장 큰 변화가 바로 이 부분입니다. 네 단계 파이프라인을 모두 설명드렸으니, 이제 이 시스템이 실제로 무엇을 할 수 있는지, Results 섹션으로 넘어가겠습니다.

Slide 17. Supported Calculation Types and Session Management

(구버전 대응: Slide 19)

이제 Results 섹션입니다. 현재 시스템이 다루는 기능부터 보겠습니다.

현재 시스템이 다루는 기능은 한두 개가 아닙니다. Orbital preview, ESP mapping, partial charge analysis, geometry optimization, single-point 수준의 결과 패키징, session continuation, clarification flow, explanation support 같은 기능이 들어와 있습니다.

이게 중요한 이유는 플랫폼 가치가 특정 기능 하나보다 연속적인 질문 흐름을 얼마나 잘 받쳐주느냐에 있기 때문입니다. 실제 연구에서는 HOMO만 보고 끝나지 않습니다. 그다음 ESP도 보고 싶고, 구조도 최적화하고 싶고, 같은 맥락에서 질문을 이어가고 싶습니다. 이 프로젝트는 바로 그 연속성을 지원하는 방향으로 설계되어 있습니다. 이 기능들이 실제로 어떻게 쓰이는지, 대표적인 사례를 보여드리겠습니다.

Slide 18. Case Study: Orbital, Geometry Optimisation, and ESP Workflows

(구버전 대응: Slide 20)

대표 예시는 세 가지로 설명할 수 있습니다. 첫째, HOMO/LUMO는 가장 직관적인 예시입니다. 전자 구조 정보가 바로 화학적 직관으로 이어지기 때문입니다. 둘째, geometry optimization은 화려한 결과보다 더 중요한 기초 작업입니다. 해석의 기준 구조를 세우기 때문입니다. 셋째, ESP는 많은 실험자에게 특히 설명력이 좋은 결과입니다. 표면 전하 분포를 색으로 보여주기 때문에, 상호작용 가능성이나 전자 밀도 차이를 빠르게 읽을 수 있습니다.

이 세 가지 예시는 각각 다른 종류의 가치를 보여줍니다. HOMO는 반응성 직관, optimization은 구조 신뢰성, ESP는 표면 해석입니다. 즉, 이 플랫폼은 예쁜 그림 한 장을 만드는 데서 그치지 않고, 서로 다른 종류의 화학적 질문을 하나의 인터페이스에서 다룰 수 있는 기반을 제공합니다. 그러면 이 사례들이 실제 시스템에서 어떻게 이어지는지, 워크플로 시연을 보여드리겠습니다.

Slide 19. Workflow Demonstration

(구버전 대응: Slide 26)

여기서는 한두 개의 짧은 예시로 실제 사용 흐름을 보여드리겠습니다. 예를 들어 HOMO 요청으로 시작한 뒤, 같은 분자에 대해 ESP나 optimization으로 이어지는 과정을 보시면, 하나의 질문이 다음 질문으로 자연스럽게 이어지는 모습을 확인하실 수 있습니다.

여기서 보실 핵심은 화려한 효과가 아니라 연속성입니다. "한 번 질문하고 끝나는 검색창"이 아니라, 같은 연구 맥락 안에서 후속 질문이 이어지는 계산 도구라는 점을 보여드리려는 예시입니다. 시연을 보셨으니, 이제 이 프로젝트가 어떤 의미를 가지고 어디로 갈 수 있는지, 마지막 섹션으로 넘어가겠습니다.

Slide 20. Implications for Laboratory Practice and Computational Literacy

(구버전 대응: Slide 21 + 22)

마지막으로 Conclusion and Outlook 섹션입니다.

이 프로젝트의 핵심 가치는 단지 계산 하나를 편하게 만드는 것이 아니라, 연구실에서 계산을 소비하는 방식 자체를 바꿀 수 있다는 데 있습니다. 실험자는 계산에 더 자주 접근할 수 있고, 질문에서 결과까지의 왕복이 짧아질 수 있습니다. 계산 전문가는 반복 요청 처리보다 더 고차원적인 검증과 해석에 시간을 쓸 수 있습니다. 연구실 전체의 계산 활용 방식이 "소수 전문가에게 의존하는 구조"에서 "실험자도 초기 탐색을 더 자율적으로 할 수 있는 구조"로 이동할 수 있습니다.

교육적 효과도 중요합니다. 실험자가 계산 결과를 더 자주 보고, 더 자주 질문하면, 계산 개념 자체에 대한 감각도 자연스럽게 커집니다. HOMO가 뭔지, ESP가 뭘 말하는지, optimization이 왜 필요한지 같은 개념이 더 이상 멀리 있는 계산화학자의 언어가 아니게 됩니다. 협업 구조도 바뀔 수 있습니다. 계산 화학자는 반복적인 요청 처리보다 더

어려운 해석과 검증 기준 설계에 집중할 수 있고, 실험자는 초기 아이디어 단계에서 계산적 근거를 더 빨리 붙일 수 있습니다. 이 플랫폼은 단순한 도구를 넘어, 연구실 안의 계산 문화와 협업 구조를 바꿀 수 있는 인프라 후보로 볼 수 있습니다. 그러면 이 프로젝트를 학술적으로 어떻게 자리매김할 수 있는지 말씀드리겠습니다.

Slide 21. Related Work, Novelty, and Publication Strategy

(구버전 대응: Slide 23 + 24)

학술적 새로움은 새로운 전자구조 방법을 만들었다는 데 있지 않습니다. 그보다는 계산 접근 워크플로를 새롭게 구성했다는 점에 있습니다. 자연어 인터페이스, MCP 스타일 도구 호출 구조, 오픈소스 양자화학 엔진, 웹 기반 결과 시각화를 하나의 플랫폼 경험으로 묶었다는 것이 핵심입니다. 이건 계산화학과 AI를 억지로 붙인 것이 아니라, 실제 연구자가 계산을 사용하는 경로를 재구성했다는 의미가 있습니다. 이 프로젝트는 computational chemistry, scientific workflow automation, human-AI interaction의 교차점에 놓여 있다고 볼 수 있습니다.

물론 앞으로 해야 할 것도 분명합니다. 참조 소프트웨어와의 체계적 정확도 비교, 구조 해석 단계의 품질 검증, 더 넓은 사용자 환경에서의 안정성 검증이 필요합니다. 동시에 이것은 다음 연구 질문이 분명하다는 뜻이기도 합니다. 논문화 방향은 단순히 "도구를 만들었다"가 아니라, 현재 워크플로 병목을 분석하고, 자연어 기반 계산 오케스트레이션 플랫폼이 그 병목을 어떻게 줄이는지 보여주는 방향입니다. 이 방향은 충분히 높은 수준의 저널로도 발전시킬 수 있습니다. 다만 메시지는 분명해야 합니다. 새로운 물리학을 주장하는 것이 아니라, 계산화학 접근 방식을 바꾸는 플랫폼적 기여를 보여주는 것입니다. 이제 앞으로의 확장 방향을 말씀드리겠습니다.

Slide 22. Future Work: Extension to Periodic Systems and Multi-Step Workflows

(구버전 대응: Slide 25)

앞으로의 확장 방향은 더 흥미롭습니다. 현재는 고립 분자 수준의 HOMO, LUMO, ESP, optimization 같은 핵심 질문을 다루는 데 집중하고 있지만, 후속 버전에서는 훨씬 더 까다로운 문제로 확장할 수 있습니다. 예를 들어 cluster DFT, surface DFT, periodic DFT, AIMD 같은 방향이 있습니다.

이건 단순히 기능을 많이 붙이겠다는 얘기가 아닙니다. 질문 중심 인터페이스와 실제 계산 엔진 호출 구조가 잘 자리 잡으면, 뒤 계산 엔진과 워크플로를 더 복잡한 시스템으로 넓힐 수 있다는 뜻입니다. 현재 플랫폼은 끝난 결과물이 아니라, 더 큰 계산 자동화 생태계로 커질 수 있는 출발점입니다. 이제 전체 발표를 정리하겠습니다.

Slide 23. Summary and Open Discussion

(구버전 대응: Slide 27)

마지막으로 세 가지만 정리하겠습니다. 첫째, 이 프로젝트는 실험자가 자연어로 양자화학 계산에 접근할 수 있게 만드는 플랫폼입니다. 둘째, 핵심 가치는 LLM이 계산을 대신하는 것이 아니라, LLM과 MCP 스타일 구조를 이용해 실제 계산 도구를 책임 있게 호출하게 만든다는 점입니다. 셋째, 이 플랫폼은 연구실 차원에서 계산의 접근성과 활용 빈도를 높이고, 장기적으로는 계산 문화를 바꿀 수 있는 잠재력을 가집니다.

토론에서는 세 가지를 함께 이야기해 보면 좋겠습니다. 실제 연구실에서 가장 먼저 자동화 가치가 큰 질문은 무엇인지, 어떤 검증 기준이 가장 중요할지, 그리고 어떤 후속 기능이 가장 우선순위가 높을지입니다. 이 발표가 단순한 결과 보고를 넘어, 다음 연구 방향을 함께 정하는 출발점이 되기를 기대합니다. 감사합니다.